HPC MPI Assignment

Michael Durkan – 17378383 – PHYS4004

17378383@student.curtin.edu.au

Introduction

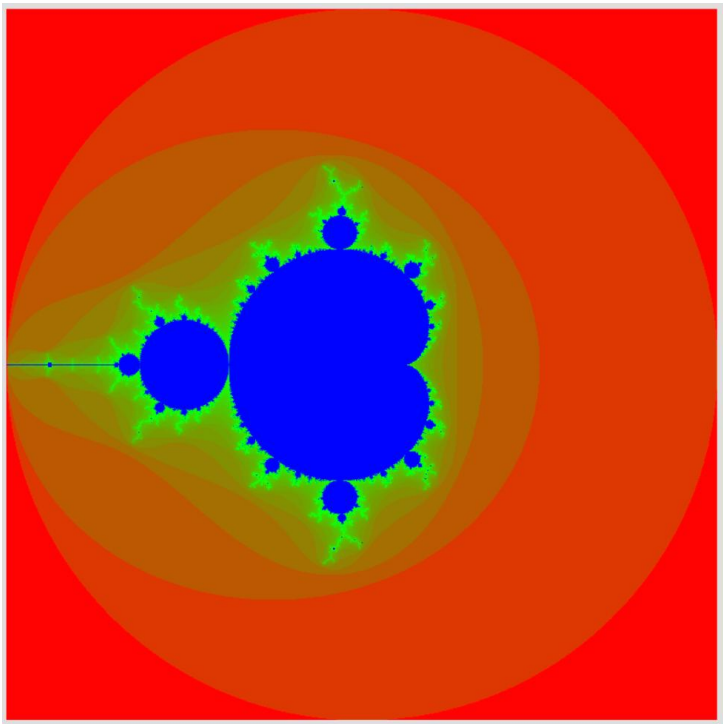
Mandelbrot Set & Serial Algorithm

The Mandelbrot set is a two-dimensional set defined in the complex plane as the complex numbers κ for which the sequence $z_{i+1} = (z_i)^2 + \kappa$ does not diverge to infinity when iterated starting at $z_0 = \kappa$. This means a point κ is in the Mandelbrot set if the given sequence is bounded as $i \rightarrow \infty$.

The serial algorithm to compute the Mandelbrot set functions by iterating over an N x N grid of pixels, representing a region of the complex plane. It converts the grid positions into complex κ values, ranging from -2 to 2 on the real & imaginary axes. A while loop then continuously calculates $z = z^2 + \kappa$, starting with a value of z = κ.

This while loop continues as long as |z| $\leq$ 2 and the iteration counter does not exceed the max iteration limit. It is a proven mathematical property that the Mandelbrot set is fully bounded within a disk of radius 2, hence when magnitude exceeds 2 the point has diverged and is not in the set. If the counter reaches a set max iteration whilst |z| $\leq$ 2 it is decided that the point remains bounded and lies within the Mandelbrot set. When this loop terminates, logarithmic scaling is applied to the final iteration count k and is stored in a 2D array. These values are converted into false colours to output the Mandelbrot set fractal.

Mandelbrot Set Render



Time Taken for Serial Code Render

Through use of the <sys/time.h> library in c it was determined that the calculation time of the serial code, without image saving, was **13.65 seconds.** The time when increasing the image size to 8000x8000, that used in the parallel programs, was **237.31 seconds**

Static Decomposition (Block Partitioning)

Data Distribution Method

In static decomposition with block partitioning the 1D pixel array is separated into equal sized, contiguous chunks, with each corresponding to horizontal rectangular strips of the complex plane. Every process is assigned one of these chunks based on its MPI rank, ensuring the workload is distributed during computation.

Tasking/Communication Algorithm

Each process knows what work to do through the using its rank along with the current iteration of the N^2 pixel loop to calculate an offset. The calculation of i = loop % N and j = (start + loop) / N within this loop determines the correct global coordinates. Once the calculation has completed the MPI_Gather function collects these chunks and assembles them back in rank order into a single array on process 0, enabling the output image to be formatted correctly.

Code with Changes

1. The output was verified using diff against the serial version, confirming byte-identical PPM files.

```c
#define MAXITER 1000
#define N   8000
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <complex.h>
#include <mpi.h>

int main(int argc, char *argv[]) {
    int    i, j, k, loop, rank, size, strip_size, start, end;
    short  green, blue;
    float  *x, *x_all = NULL;
    FILE   *fp;
    float complex   z, kappa;

    MPI_Init(&argc,&argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    strip_size = (N*N) / size;
    start = strip_size * rank;

    x = (float *) malloc(strip_size*sizeof(float));
    if (rank == 0) {
        x_all = (float *) malloc((long)N * N * sizeof(float));
    }
    for (loop = 0; loop < strip_size; loop++) {
            // i:cols and j:rows
            i = (start + loop) % N;
            j = (start + loop) / N;
            kappa= (4.0*(i-N/2))/N + (4.0*(j-N/2))/N * I;
            z=kappa;
            k=1;

            while ((cabs(z)<=2) && (k++<MAXITER)) {
                z= z*z + kappa;
            }
            x[loop]= log((float)k) / log((float)MAXITER);
    }
```

```
    if (rank == 0) {
        printf("Writing mandelbrot2.ppm\n");
        fp = fopen ("mandelbrot2.ppm", "w");
        fprintf (fp, "P6\n%4d %4d\n255\n", N, N);

        for (loop=0; loop<N*N; loop++)
            if (x_all[loop]<0.5) {
                green= (int) (2*x_all[loop]*255);
                    fprintf (fp, "%c%c%c", 255-green,green,0);
            } else {
                blue= (int) (2*x_all[loop]*255-255);
                    fprintf (fp, "%c%c%c", 0,255-blue,blue);
            }
        fclose(fp);
        free(x_all);
    }
    free(x);
    MPI_Finalize();
    return 0;
```

Time Taken for Block Partitioning Render & Speed-Up

Total time to complete calculation: This is equal to the process that took the longest, which was rank 4, at **78.4 seconds.**

Speedup relative to serial (S) = Time for serial code compute / time for parallel code compute

= T_serial / T_parallel ≈ 237.3 / 78.4 = **3.03**

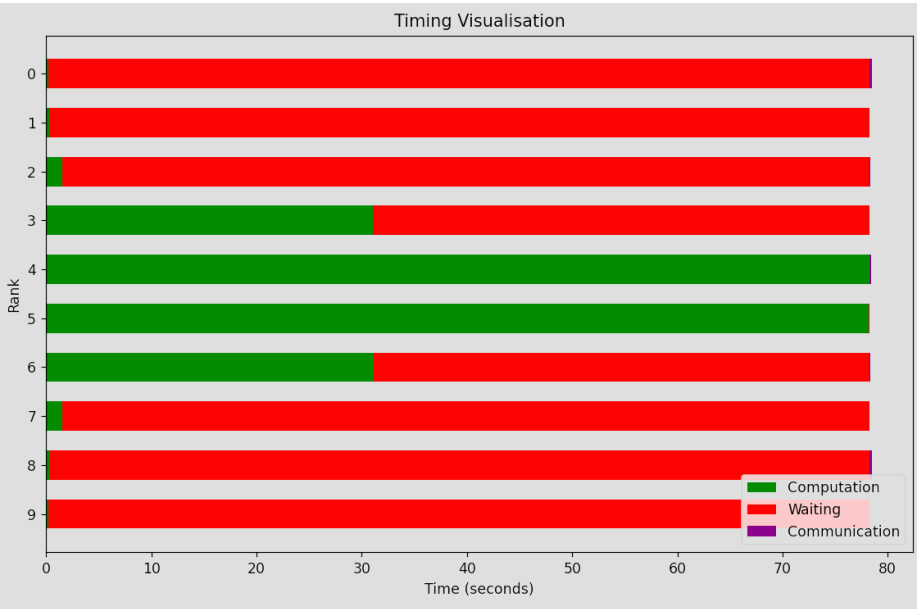
Ideal Speed (P) = Amdahl's Law with serial fraction = 0, P = **10**

Efficiency (E) = S / P = 3.03 /10 = **30.3%**

Results:

Rank 9 : comp = 0.2245 s wait = 78.0336 s comm = 0.0352 s
Rank 3 : comp = 31.0647 s wait = 47.1935 s comm = 0.0360 s
Rank 5 : comp = 78.2181 s wait = 0.0400 s comm = 0.0360 s
Rank 1 : comp = 0.2994 s wait = 77.9587 s comm = 0.0361 s
Rank 7 : comp = 1.5690 s wait = 76.6891 s comm = 0.0363 s
Rank 2 : comp = 1.5564 s wait = 76.7017 s comm = 0.0790 s
Rank 6 : comp = 31.1425 s wait = 47.1156 s comm = 0.0797 s
Rank 4 : comp = 78.2581 s wait = 0.0000 s comm = 0.1653 s
Rank 0 : comp = 0.2239 s wait = 78.0342 s comm = 0.2071 s
Rank 8 : comp = 0.3002 s wait = 77.9579 s comm = 0.2077 s

Timing Figure



Load Balancing Evaluation

The timing figure shows the extreme load imbalance across the 10 processes. There is a maximum difference of over 250x for completion time, with ranks 4 and 5 spending 78 seconds on computation, whilst ranks 0, 1, 8 and 9 finish in under 0.3 seconds. Those that finish quickly waste their allocated CPU time sitting idle.

This is due to the difference in workload across the complex plane of the Mandelbrot set, when tackled with a static block partitioning strategy. The centre of the plane contains the highest concentration of pixels that require close to the max iteration cap, whilst the edges are furthest from the set and escape the loop almost immediately. It is also worth noting that the symmetry of the Mandelbrot set is directly reflected in the mirrored workloads of these processes. In conclusion, the workload was not evenly distributed between the processes, creating a bottleneck resulting in poor parallel efficiency of 30.3%.

Dynamic Decomposition (Master-Worker)

Data Distribution Method

In a dynamic Master-Worker Decomposition the master rank 0 process hands out the work as needed, instead of deciding at the beginning. This master distributes equal sized chunks of work to the workers on demand. It starts by sending each worker a chunk and then enters a loop where it waits for any available worker to return results via MPI_ANY_SOURCE. Once it receives this it assigns the worker the next chunk. This ensures that workers never sit waiting too long, balancing the workload.

Tasking/Communication Algorithm

The algorithm used MPI_Send and MPI_Recv, point to point communication, instead of collective routines such as MPI_Gather. Each task has a starting pixel index, which is sent to a ready worker. This worker computes that chunk and sends the index back with the calculated result. The master then places this data in the final output array. The master keeps track of the remaining work and increments the starting index after each work assignment. Once the workload is complete the master replies to incoming results with an index greater than the total pixels, which the workers recognise as a signal to terminate their loop.

Code with Changes

1. The output was verified using diff against the serial version, confirming byte-identical PPM files.
2. The code was compiled with -g -fsanitize=address to verify no memory access faults

```c
    // Initilization and repeated MPI init from task 1
    if (argc < 2) {:
        if (rank == 0) {
            printf("Program execution: %s <chunksize>\n", argv[0]);
            MPI_Finalize();
            return 0;
        }
    }

    chunksize = atoi(argv[1]);
    num_workers = size - 1;

    // Master
    if (rank == 0) {
        x = (float *)malloc(total_pixels * sizeof(float));
        recv_buf = (float *)malloc(chunksize * sizeof(float));
        next_start = 0;
        finished_workers = 0;

        for (w = 1; w <= num_workers; w++) {
            MPI_Send(&next_start, 1, MPI_INT, w, 0, MPI_COMM_WORLD);
            next_start += chunksize;
    } else {
        MPI_Send(&total_pixels, 1, MPI_INT, w, 0, MPI_COMM_WORLD);
        finished_workers++;
    }

        while (finished_workers < num_workers) {
            MPI_Recv(&recv_index, 1, MPI_INT, MPI_ANY_SOURCE, 0, MPI_COMM_WORLD, &status);
            src = status.MPI_SOURCE;
            MPI_Recv(recv_buf, chunksize, MPI_FLOAT, src, 1, MPI_COMM_WORLD, &status);

            for (loop = 0; loop < chunksize && (recv_index + loop) < total_pixels; loop++)
                x[recv_index + loop] = recv_buf[loop];

            if (next_start < total_pixels) {
                MPI_Send(&next_start, 1, MPI_INT, src, 0, MPI_COMM_WORLD);
                next_start += chunksize;
            } else {
                MPI_Send(&total_pixels, 1, MPI_INT, src, 0, MPI_COMM_WORLD);
                finished_workers++;
            }
        }
        // Repeated image output code from Task 1
        fclose(fp);
        free(x);
        free(recv_buf);

    } else {
        // Worker
        buf = (float *)malloc(chunksize * sizeof(float));
        MPI_Recv(&start, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, &status);

        while (start < total_pixels) {
            for (loop = 0; loop < chunksize; loop++) {
                idx = start + loop;

                if (idx < total_pixels) {
                    // Repeated Mandelbrot calculations from task 1
                    buf[loop] = log((float)k) / log((float)MAXITER);
                } else {
                    buf[loop] = 0.0f;
                }
            }

            MPI_Send(&start, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);
            MPI_Send(buf, chunksize, MPI_FLOAT, 0, 1, MPI_COMM_WORLD);
            MPI_Recv(&start, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, &status);
        }
        free(buf);
    }
    MPI_Finalize();
    return 0;
}
```

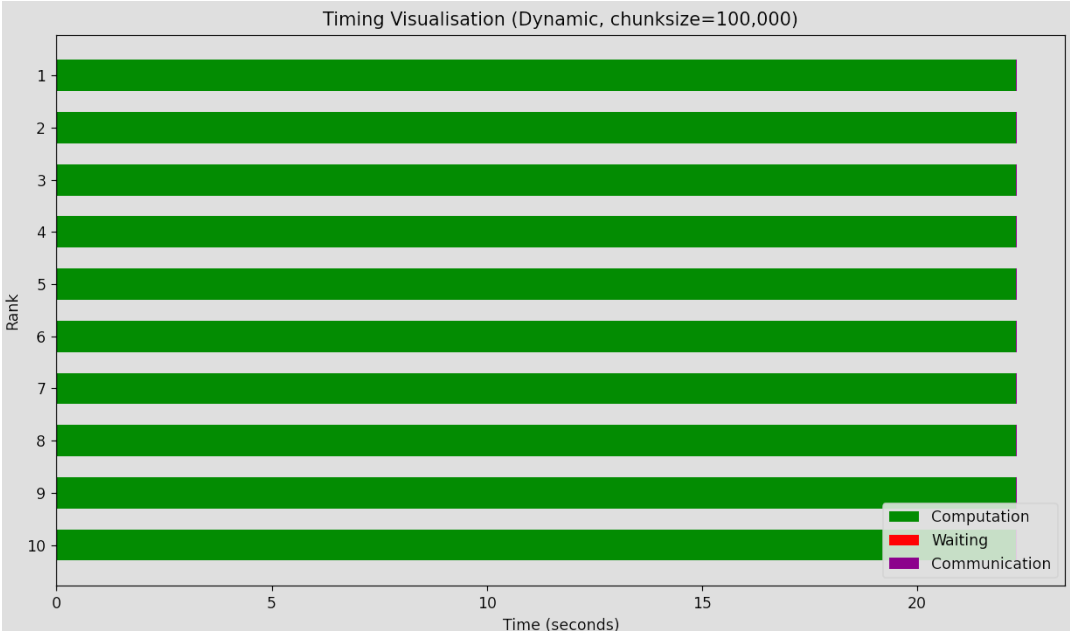
Time Taken for Master-Worker Render & Speed-Up

Total time to complete calculation: The master computed in 22.30 seconds, representing the total compute time.
Speedup relative to serial (S) = Time for serial code compute / time for dynamic compute
= T_serial / T_parallel ≈ 237.3 / 22.30 = **10.63**
Ideal Speed (P) = Amdahl's Law with serial fraction = 0, P = **11**
Efficiency (E) = S / P = 10.63 /11 = **96.6%**

Results

Rank 2: comp = 22.3017 s, comm = 0.0198 s
Rank 1: comp = 22.3002 s, comm = 0.0214 s
Rank 8: comp = 22.3000 s, comm = 0.0221 s
Rank 5: comp = 22.3040 s, comm = 0.0182 s
Rank 9: comp = 22.3008 s, comm = 0.0219 s
Rank 10: comp = 22.3046 s, comm = 0.0184 s
Rank 6: comp = 22.3040 s, comm = 0.0192 s
Rank 3: comp = 22.3043 s, comm = 0.0192 s
Rank 4: comp = 22.3049 s, comm = 0.0188 s
Master: total = 22.3240 s
Rank 7: comp = 22.3036 s, comm = 0.0203 s

Timing Figure



Load Balancing Evaluation

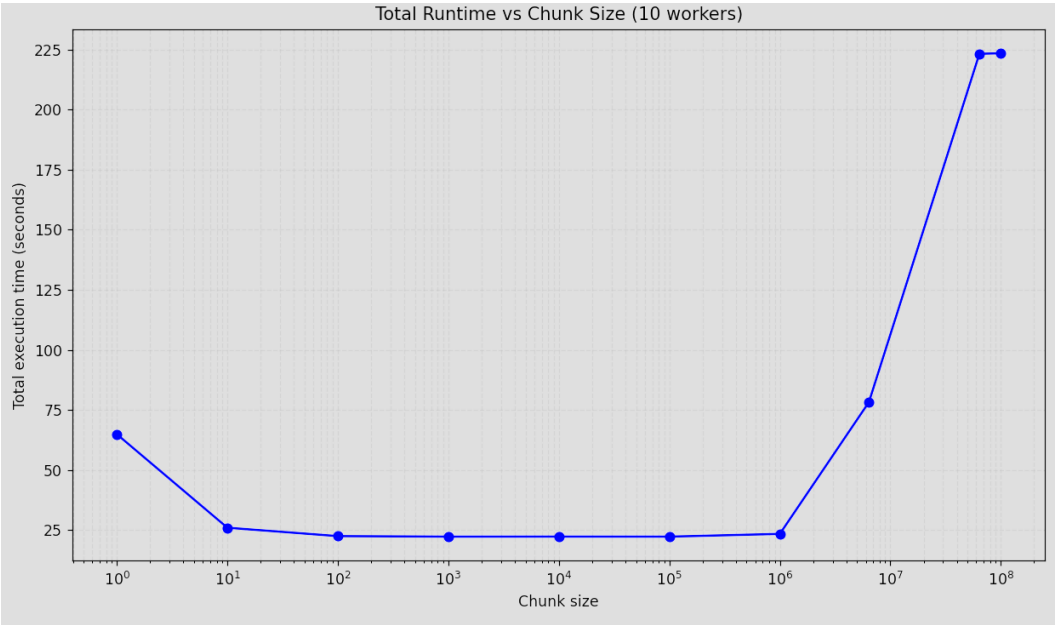
The load balancing across the worker processes is very close to perfectly even. Each worker spends around 22.3 seconds on computation, with very small variation of less than 0.005 seconds between them. The communication overhead is also negligible (~0.02 seconds).

This great performance is a result of the dynamic master-worker distribution, where unlike block partitioning used in task 1, small chunks of work are assigned on demand rather than at the beginning. Workers that receive easier tasks, those nearer to edges of the set, complete the work quickly and then receive more chunks to process. This creates a self-balancing effect on the workload.

This dynamic version reduces the imbalance from task 1 to eliminate idle waiting time, taking the efficiency from ~30% to an efficiency of ~96.6%

Total Runtime as Function of Chunk Size

Figure



Explanation

At small chunk sizes of 1-10 the execution time is elevated, with a size 1 chunk taking 65 seconds. Computation per worker is stable at ~22.5 seconds but the master becomes a bottleneck, as when the chunks are this small the volume of MPI_Send and MPI_Recv for it to handle becomes massive, causing the workers to spend a lot of time waiting to communicate.

In the optimal range of 100 to 100,000 pixels per chunk the execution drops to a minimum stable value of 22.3 seconds. The chunks are large enough for negligible communication overhead, yet small enough for the dynamic scheduler to effectively balance the nonuniform workload across all 10 workers.

At large chunk sizes of 1,000,000 and above the execution time rises sharply as the scheduler can no longer distribute the work evenly. Above the value of 64,000,000 a single worker processes the entire image whilst the rest sit idle, increasing the time to compute back to serial time of ~237 seconds.

Specific Chunk Size Comparison with Static Block Partitioning

For a chunk size equal to one tenth of the total workload, 6,400,000 pixels, the completion time is 78.3 seconds, which is nearly identical to the 78.5 seconds of the static block partition from Task 1. This is to be expected as with 10 chunks and workers the workload is divided into 10 even chunks like in static block partitioning. The scheduler gets no opportunity to redistribute the work, so the workers process the same amounts of rows as the static assignment. This results in the same severe load imbalance, with the more complicated workload ranks taking 78 seconds whilst the fast ranks taking less than a second.

Static Decomposition (Cyclic Partitioning)

Data Distribution Method

In static decomposition with cyclic partitioning the pixel array is distributed round robin rather than in contiguous blocks. Each process evaluates pixels separated by the amount of total processes. This allows each process to receive an equal mix of computationally cheap edge, and expensive centre pixels. This handles load balancing without the need for a master process.

Tasking/Communication Algorithm

Each rank determines its pixel workload from the cyclic formula, so no initial distribution of the data from rank 0 is needed. Once computation is complete the processes use MPI_Gather to send their local results back to the root process. This data is returned in the cyclic order however, so to construct the final image rank 0 gathers the data in a temporary buffer (y) and then undoes the cyclic formula with x[loop] = y[r * chunksize + l].

Code with Changes

```
    chunksize = total_pixels / size;

    x_local = (float *)malloc(chunksize * sizeof(float));

    for (l = 0; l < chunksize; l++) {
        loop = l * size + rank;
        i = loop % N;
        j = loop / N;

        // Mandelbrot calculations from task 1
    }

    y = NULL;
    x = NULL;
    if (rank == 0) {
        y = (float *)malloc(total_pixels * sizeof(float));
        x = (float *)malloc(total_pixels * sizeof(float));
    }

    MPI_Gather(x_local, chunksize, MPI_FLOAT, y, chunksize, MPI_FLOAT, 0, MPI_COMM_WORLD);

    if (rank == 0) {
        for (r = 0; r < size; r++) {
            for (l = 0; l < chunksize; l++) {
                loop = l * size + r;
                x[loop] = y[r * chunksize + l];
            }
        }

        // Writing image to file code from task 1

        fclose(fp);

        free(x);
        free(y);
    }

    free(x_local);
    MPI_Finalize();
    return 0;
}
```

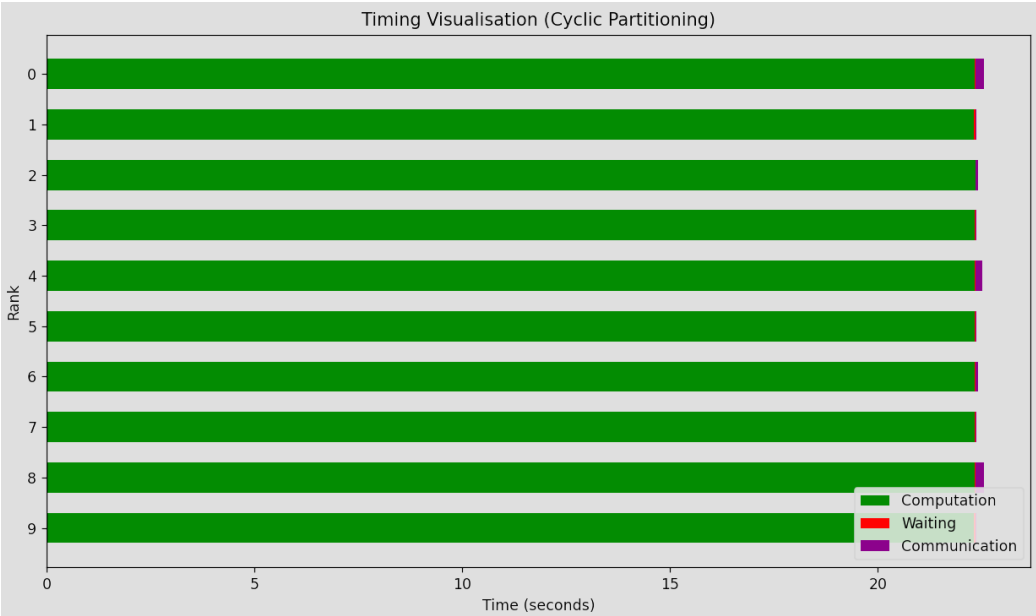
Time Taken for Cyclic Partitioning Render & Speed-Up

Total time to complete calculation: Rank 0 completed in 22.55 seconds, representing the total compute time.
Speedup relative to serial (S) = Time for serial code compute / time for cyclic compute
= T_serial / T_parallel ≈ 237.3 / 22.55 = 10.52
Ideal Speed (P) = Amdahl's Law with serial fraction = 0, P = **10**
Efficiency (E) = S / P = 10.52 /10 = **105.2%**

Results

```
Rank  1 : comp =  22.3184 s  wait =   0.0214 s  comm =   0.0369 s
Rank  5 : comp =  22.3292 s  wait =   0.0106 s  comm =   0.0371 s
Rank  3 : comp =  22.3347 s  wait =   0.0052 s  comm =   0.0374 s
Rank  7 : comp =  22.3256 s  wait =   0.0143 s  comm =   0.0376 s
Rank  9 : comp =  22.3177 s  wait =   0.0222 s  comm =   0.0378 s
Rank  2 : comp =  22.3398 s  wait =   0.0000 s  comm =   0.0805 s
Rank  6 : comp =  22.3362 s  wait =   0.0036 s  comm =   0.0810 s
Rank  4 : comp =  22.3354 s  wait =   0.0044 s  comm =   0.1665 s
Rank  0 : comp =  22.3240 s  wait =   0.0158 s  comm =   0.2081 s
Rank  8 : comp =  22.3290 s  wait =   0.0109 s  comm =   0.2088 s
```

Timing Figure



Load Balancing Evaluation

The timing figure shows even load balancing across the 10 processes. Every rank spends 22.33 seconds on computation, with a difference of 0.02 seconds between the fastest and slowest. The wait time is negligible, and the communication time is very small 0.02-0.21 seconds. No process sits idle for a significant amount of time.

This is due to the assigning of every 10th pixel to each rank in the round robin fashion, where every process gets an even amount of pixels across the different difficulties on the complex plane. This distribution of cheap and expensive pixels ensures the computation cost is equal without runtime scheduling.

Comparison of Timing and Load-Balancing with Block Partitioning

Cyclic results in an improvement of timing and load balancing when compared to task 1. In block partitioning the contiguous distribution results in a severe bottleneck, with the dense central rows taking over 78 seconds to complete, whilst outer edges completed in under a second. With cyclic partitioning the heavy central workload is evenly shared amongst the 10 processes, causing the execution time to drop from ~78.5 seconds to ~22.5 seconds, a 3.5x improvement with the same number of processes.

Conclusions

Overall Comparisons of Methods

Static Block Partitioning, Dynamic Master Worker, and Static Cyclic Partitioning were evaluated for parallelising the Mandelbrot set. The non-uniform computational cost across the complex plane produced a significant difference between the effectiveness of the methods. Block Partitioning had severe load imbalance due to the more complicated pixel calculations being distributed between only a few of the ranks, whilst Dynamic and Cyclic partitioning eliminated this bottleneck, reducing the compute time from ~78.5s to ~22.5s. Runtime scheduling was used by the dynamic approach to complete this, whilst mathematical interleaving was the method for cyclic partitioning.

	Block Partition	*Dynamic*	*Cyclic Partitioning*
Total Time	~78.5s	~22.3s	~22.5s
Speedup	3.03	10.63	10.52
Efficiency	30.3%	96.6%	105.2%
Load balancing	Large Imbalance (250x)	Close to perfect	Close to perfect
Communication	Single MPI_Gather	Continuous Point-to-Point	Single MPI_Gather + reorder
Num Processes	10	10 workers + 1 master	10

Strengths and Weaknesses of Each Method

Static Decomposition - Block Partitioning

Strengths

Block Partitioning is the simplest to code, with just a division of rows to be performed, and an MPI_Gather that requires no reordering. It leaves the network unused during calculation and is useful when computation cost is uniform across the problem domain.

Weaknesses

When the workload is not uniform, like in the Mandelbrot set, it creates a severe load imbalance. When the contiguous blocks of data are set over areas with fluctuating workloads, like the dense fractal centre and the quick to compute edges, the edge processes will sit idle. This led to poor parallel efficiency of 30.3% for this problem.

Dynamic Decomposition - Master-Worker

Strengths

Dynamic Decomposition creates near perfect load balancing regardless of the workload's distribution. It adapts on the fly so that idle workers receive more work when they are ready. Through tuning the chunk size, a near ideal speedup was achieved.

Weaknesses

This method requires a dedicated process for scheduling, which meant that 11 processes are needed to do the work that 10 did in the other methods. It also needs point-to-point communication that can bottleneck the network, especially with small chunk sizes. It is also the most difficult to implement, with care needed for messaging and manual array creation.

Static Decomposition – Cyclic Partitioning

Strengths

Cyclic partitioning achieves excellent load balance through interleaved pixel assignment while all processes compute. The communication overhead is minimal with just a single MPI_Gather, and it is also simpler than the dynamic approach to implement. It achieved near ideal speedup at ~10.5x.

Weaknesses

A restructuring loop is required after the MPI_Gather due to the data being interleaved. It could also lead to load imbalance if there were regular intervals of workload difficulty across the domain. It therefore could perform worse than the dynamic approach for unpredictable or randomised problem sets.

Best Performing Method

The best performing method for generating the Mandelbrot set was cyclic partitioning, as can be seen by its similar compute time to the dynamic approach, with 1 less processor. It matches the dynamic version's load balance and speedup, whilst remaining structurally simpler. It utilises all available processes for computation, meaning a dedicated process isn't needed for the master rank, leading to the highest efficiency of the methods of ~105%. It also only requires one MPI_Gather instead of the continuous network messages of the dynamic approach. The overhead of reordering the array is negligible compared to the total compute time.

Whilst the dynamic approach was equally fast its complexity was not required for a problem like this where the non-uniform cost was predictable. Dynamic decomposition would be better suited for a problem where cost distribution is irregular or unknown, but for the Mandelbrot set, the regularity of the workload makes cyclic partitioning the best strategy.